

Vehicle Parking Management System - Project Report

Student Details

Name: Somya Menariya

Roll Number: 24f1002734

Course: MAD-1 Project

Email: 24f1002734@ds.study.iitm.ac.in

Institute: IIT Madras

Greetings everyone I am Somya Menariya, from Banswara, Rajasthan. I'm pursuing the BS in Data Science and Programming from IIT Madras. I enjoy building real-world applications and solving practical problems.

Problem Statement

This web-based application helps manage 4-wheeler parking lots. Admins can manage lots/spots, while users can reserve or release spots. It reduces congestion, improves utilization, and maintains parking history.

Approach

The app uses Flask (backend), SQLAlchemy (ORM), SQLite (database), Jinja2 (templating), and Bootstrap (UI). Role-based logic, modular routes, validation, and time-based billing are implemented.

Key Features

- Role-based login (Admin/User)
- Admin can manage lots and spots
- Auto-spot creation
- Real-time status updates
- Parking cost calculation
- Reservation history
- Search by fields (admin)
- Responsive Bootstrap UI

Technologies Used

- Flask: Backend web framework
- Flask-SQLAlchemy: ORM for model handling
- SQLite: Lightweight database
- Jinja2: HTML rendering engine
- Bootstrap 5: Responsive styling
- Dotenv: Manage app configuration

This Vehicle Parking Management System (V1) project was developed with the assistance of **AI-based tools** for code formatting, debugging, and template guide.

- **Percentage of AI Assistance Used in Development: Approximately 40%**

Database Schema and Table Descriptions

1. User Table

Fields:

- **id:** Primary key, unique identifier for each user.
- **email:** User's email address, must be unique and not nullable.
- **password:** User's password, not nullable.
- **name:** Full name of the user, not nullable.
- **address:** User's address, default is ' NA ' .
- **pincode:** User's area pin code, default is ' NA ' .
- **role:** Role of the user, default is ' user ' .

Purpose: Stores information of all users and identifies their access level.

2. ParkingLot Table

Fields:

- **id:** Primary key, unique identifier for each parking lot.
- **prime_location_name:** Name of the main location/area for the lot.
- **price:** Parking price per hour (default 100).
- **address:** Address of the parking lot, default is ' NA ' .
- **pincode:** Pincode of the lot, default is ' NA ' .
- **maximum_number_of_spots:** Maximum number of spots allowed in the lot.

Purpose: Represents parking lot infrastructure managed by the admin.

3. ParkingSpot Table

Fields:

- **id:** Primary key, unique identifier for each spot.
- **lot_id:** Foreign key referencing ParkingLot .id.
- **status:** Enum field, either ' AVAILABLE ' or ' OCCUPIED ' (default is ' AVAILABLE ').

Purpose: Tracks the status and availability of each spot in a parking lot.

4. Reservation Table

Fields:

- **id:** Primary key, unique reservation identifier.
- **user_id:** Foreign key referencing User .id.
- **lot_id:** Foreign key referencing ParkingLot .id.
- **spot_id:** Foreign key referencing ParkingSpot .id.
- **vehicle_number:** User's vehicle registration number.
- **start_time:** Reservation start timestamp (default is current time).
- **end_time:** Reservation end timestamp (nullable).
- **cost:** Total parking cost calculated based on time and lot price.

Purpose: Records each parking activity and helps in cost calculation and history.

Table Relationships

- User → Reservation: One-to-Many
- ParkingLot → ParkingSpot: One-to-Many
- ParkingSpot ↔ Reservation: One-to-One
- Reservation → ParkingLot: Many-to-One

Project Video:

[For video click here](#)

ER Diagram:

[click here](#)